



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Universidad Nacional de Colombia - sede Bogotá
Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Curso: Ingeniería de Software

Estudiantes: Camilo Andres Lievano Rendon, Juan Mateo Rozo Torres, Juan Diego Lasso Montaña, Rene Alejandro Barrero Gonzalez.

Tests Unitarios:

test_jrozot_CU20.py

```
1 2 3 4 5 6 7 8 9 []= juanmateo@LAPTOP-DJ5Z04T:~/src/Ing_Software_I/Ingesoft2025-168/Proyecto/tests
[jm@T480]$python -m unittest test_jrozot_CU20.py
.....
-----
Ran 5 tests in 0.031s

OK
[jm@T480]$
```

Este test verifica la funcionalidad de la pantalla de cambio de contraseña como una ventana de aplicación basada en PyQt5. Se concentra en la validación de la contraseña ingresada, la lógica de actualización, la integración con la base de datos y la respuesta al usuario.

Validación de datos ingresados:

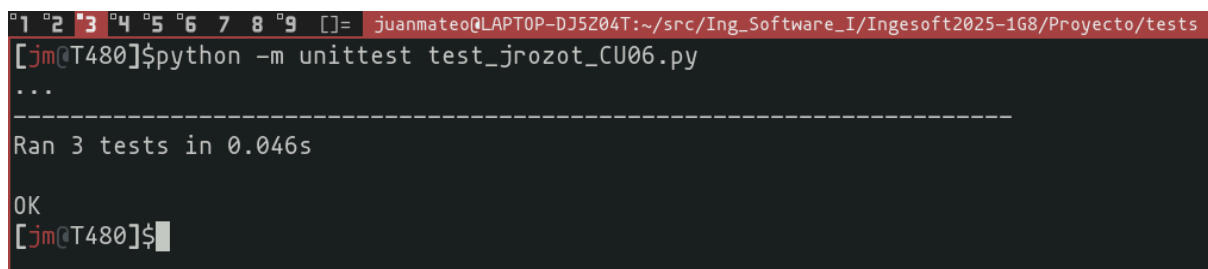
1. **test_empty_fields:** Se asegura de que los usuarios sean informados cuando los campos requeridos se dejan vacíos.
2. **test_invalid_current_password:** Verifica que el sistema detecte cuando la contraseña actual ingresada no es válida.
3. **test_passwords_do_not_match:** Comprueba que se muestre un error cuando la nueva contraseña y su confirmación no coinciden.
4. **test_invalid_new_password:** Valida que la nueva contraseña cumpla con los requisitos de seguridad (longitud, mayúsculas, números, símbolos).

Actualización y persistencia en la base de datos:

1. **test_successful_password_change**: Simula una actualización exitosa de la contraseña, verifica que se actualice el hash en la base de datos, se registre en el historial y se notifique correctamente al usuario.

Estas pruebas son importantes para garantizar la seguridad, consistencia y experiencia de usuario en el sistema de autenticación.

test_jrozot_CU06.py



```
1 2 3 4 5 6 7 8 9 []= juanmateo@LAPTOP-DJ5Z04T:~/src/Ing_Software_I/Ingsoft2025-1G8/Proyecto/tests
[jm@T480]$python -m unittest test_jrozot_CU06.py
...
-----
Ran 3 tests in 0.046s

OK
[jm@T480]$
```

Este conjunto de pruebas verifica el comportamiento de la pantalla de inicio de sesión (LoginScreen), una interfaz PyQt5 que permite a los usuarios autenticarse en el sistema. Las pruebas se centran en la validación de entradas del usuario, la lógica de autenticación, la integración con la base de datos y la respuesta visual apropiada en la interfaz.

Validación de campos ingresados:

1. **test_missing_fields**: Asegura que el sistema muestre un mensaje de advertencia cuando uno o ambos campos (correo y contraseña) estén vacíos.

Lógica de autenticación:

1. **test_successful_login**: Simula un inicio de sesión exitoso, verificando que las credenciales coincidan y que se invoque el callback para continuar con la aplicación.
2. **test_invalid_credentials**: Verifica que el sistema rechace las credenciales inválidas y muestre el mensaje de error correspondiente cuando no se encuentra un usuario válido o su contraseña no es correcta.

Importancia:

Estas pruebas son esenciales para garantizar que el sistema solo permita el acceso a usuarios autorizados, proporcione retroalimentación clara ante errores, y mantenga una experiencia de usuario consistente desde la autenticación, uno de los puntos más críticos en cualquier aplicación.

test_jrozot_main_window.py

```

1 2 3 4 5 6 7 8 9 []= juanmateo@LAPTOP-DJ5Z04T:~/src/Ing_Software_I/Ingesoft2025-168/Proyecto/tests
[jm@T480]$python -m unittest test_jrozot_main_window.py
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26dcd940>
Rol identificado: Administrador
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26e09590>
Rol identificado: Revisor
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26e0b610>
Rol identificado: Restaurador
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26eb02b0>
Rol identificado: Digitalizador
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26eb22c0>
Rol identificado: Supervisor de calidad
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26ea0ef0>
Rol identificado: Clasificador
Usuario leído: <test_jrozot_main_window.MockUser object at 0x7f1b26dbd6a0>
Rol identificado: Lector
.
-----
Ran 1 test in 0.281s

OK
[jm@T480]$

```

Este conjunto de pruebas valida la correcta asignación de permisos en la ventana principal (MainWindow) según el rol del usuario autenticado. Cada rol en el sistema tiene acceso a un conjunto específico de funcionalidades, lo cual se refleja en la interfaz mediante una lista de características habilitadas.

Asignación de permisos por rol:

1. **test_permissions_by_role**: Verifica que cada tipo de usuario (por ejemplo, Administrador, Revisor, Restaurador, etc.) reciba exactamente las funcionalidades que le corresponden según la definición del sistema. Esto se realiza simulando un usuario con un rol determinado y comprobando que la lista de **available_features** coincida con las funcionalidades esperadas para ese rol.

Importancia:

Estas pruebas son fundamentales para garantizar que la aplicación respete el control de acceso por rol. Aseguran que los usuarios solo puedan ver y utilizar las funcionalidades que les están autorizadas, lo cual es esencial para la seguridad, la

claridad de la interfaz y el cumplimiento de las responsabilidades asignadas en el flujo de trabajo del proyecto.

test_rbarrero_CU05.py

Funcionalidad validada: Clasificación de un libro existente en la base de datos por parte del usuario.

Prueba implementada:

1. test_save_classification_success:

- a. Simula el proceso de clasificación de un libro en estado "Pendiente" por otro estado nuevo ("Clasificado").
- b. Valida la modificación de atributos (`categoria_id`, `estado_id`).
- c. Verifica la escritura en el historial de cambios (bitácora).
- d. Confirma que se realiza el commit y se muestra un mensaje informativo al usuario.

2. Cobertura:

- a. Actualización de objetos ORM.
- b. Interacción con sesión de base de datos simulada.
- c. Interfaz de usuario (simulada).
- d. Registro de historial con `write_to_historial`

test_rbarrero_CU09.py

Funcionalidad validada: Proceso completo de creación de un nuevo usuario en el sistema.

Pruebas implementadas:

1. test_create_user_success:

- a. Simula ingreso de datos válidos para un nuevo usuario.
- b. Asegura que el usuario se guarda, se realiza commit y se registra en la bitácora.
- c. Verifica que se muestre un mensaje de éxito al usuario.

2. test_create_user_duplicate_email:

- a. Simula intento de crear un usuario con correo electrónico ya registrado.
- b. Verifica que no se guarda el usuario ni se realiza commit.

- c. Asegura que se muestra mensaje de advertencia al usuario.

3. Cobertura:

- a. Lógica de validación de unicidad de email.
- b. Inserción condicional en base de datos.
- c. Registro histórico y mensajes en la UI.

test_rbarrero_history_logger.py

Funcionalidad validada: Función `write_to_historial` para registrar eventos de usuario en la base de datos.

Pruebas implementadas:

1. `test_write_to_historial_success:`

- a. Verifica que la función agrega correctamente una entrada con los datos recibidos.
- b. Asegura que se llaman `session.add()` y `session.commit()`.

2. `test_write_to_historial_db_failure:`

- a. Simula una excepción al intentar hacer `commit`.
- b. Verifica que se lanza la excepción correctamente.

Cobertura:

- 1. Flujo normal y flujo de error en el registro histórico.
- 2. Interacción directa con capa de persistencia (mockeada).

test_julassom_CU04.py

```
...  
-----  
Ran 3 tests in 0.002s  
  
OK  
PS C:\Users\USER\Desktop\Ingesoft2025-1G8> 
```

Funcionalidad validada: Estos Test verifican la funcionalidad de los métodos utilizados en la pantalla de digitalización(CU04_digitize_book_screen) de libros y que se ejecuten correctamente

Pruebas implementadas:

test_guarda_digitalizacion_exitosa:

Comprueba que el método para guardar la digitalización (save_digitization) se puede llamar sin errores.

test_load_eligible_books_successfully:

Verifica que el método para cargar los libros elegibles (_load_eligible_books) se ejecuta sin errores.

test_save_digitization_file_not_found:

Comprueba que el método para guardar la digitalización (save_digitization) se puede llamar incluso si el archivo no existe, sin lanzar errores.

test_julassom_CU17.py

```
..  
-----  
Ran 2 tests in 0.001s  
  
OK  
PS C:\Users\USER\Desktop\Ingesoft2025-1G8> 
```

Funcionalidad validada: valida el comportamiento de los métodos principales de la pantalla de búsqueda de libros (CU17_search_books_screen), correspondiente al caso de uso CU17. Se busca asegurar que los componentes fundamentales relacionados con la consulta y carga de datos en esta interfaz operan sin errores y cumplen con la lógica esperada del sistema.

Pruebas implementadas:

test_busqueda_con_titulo_filtra_correctamente:

Evalúa que el método search_books, encargado de realizar búsquedas en base al

título del libro y otras características, pueda ser invocado correctamente sin lanzar excepciones. La prueba también implica que el filtrado aplicado a los resultados funciona como se espera, llenando la tabla correspondiente

test_load_combos_successfully:

Verifica que el método `_load_combos`, encargado de poblar los menús desplegables utilizados para aplicar filtros en la búsqueda, se ejecuta de forma exitosa. Se asegura que la pantalla esté debidamente inicializada con los datos requeridos para la interacción del usuario.

test_julassom_CU18.py

```
..
-----
Ran 2 tests in 0.001s

OK
PS C:\Users\USER\Desktop\Ingesoft2025-168>
```

Funcionalidad validada: Estos tests verifican que los métodos principales de la pantalla de búsqueda de usuarios (`CU18_search_users_screen`) se ejecutan correctamente.

Pruebas implementadas:

test_busqueda_llena_la_tabla:

Verifica que el método encargado de ejecutar la búsqueda de usuarios (`search_users`) puede ser invocado correctamente. Se espera que la tabla de resultados se actualice sin errores, lo que indica que el mecanismo de consulta y visualización está funcionando adecuadamente..

test_load_combos_successfully:

Evalúa que el método interno (`_load_combos`), responsable de inicializar los elementos desplegables (comboboxes) de la interfaz, se ejecute sin fallos. Esta prueba garantiza que los datos necesarios para los filtros u opciones de búsqueda se cargan correctamente al inicio o actualización de la pantalla.

test_clievano_CU01.py

Funcionalidad validada: verifica el registro de un nuevo libro al sistema.

Pruebas implementadas:

test_registrar_libro_success:

Datos de entrada desde los campos de UI (ISBN, título, autor, etc.).

Simulación de estado inicial ("Registrado").

Mock de la base de datos y la sesión de SQLAlchemy.

Validación de que se llamó a commit() y write_to_historial().

Validación de que se muestra un QMessageBox de éxito.

test_clievano_CU02.py

Funcionalidad validada: verifica el proceso del registro de condición física de un libro se complete correctamente.

Pruebas implementadas:

test_registrar_revision_exito:

El usuario puede registrar la condición física de un libro con estado "Registrado".

El sistema valida las fechas.

Se determina si el libro pasa a "Digitalización" o "Restauración".

Se actualiza el estado del libro y se guarda la tarea.

Se registra la acción en el historial.

Se muestra un mensaje de éxito.

test_clievano_CU25.py

Funcionalidad validada: verifica que el usuario pueda crear una nueva categoría en el sistema.

Pruebas implementadas:

test_crear_categoria_campos_vacios:

Verifica que se muestre el error cuando el nombre está vacío.

test_crear_categoria_nombre_largo:

Prueba la validación de longitud máxima del nombre.

test_crear_categoria_existente:

Verifica que no se permite crear categorías duplicadas.

test_crear_categoria_exito:

Prueba el flujo normal de creación, incluyendo el registro en el historial.

test_crear_categoria_error_db:

Verifica el manejo de errores durante el commit.

Uso de linters:

Informe de análisis estático con Ruff

Se utilizó la herramienta **Ruff** como analizador estático de código (linter) para revisar la calidad, estilo y consistencia del código fuente del proyecto.

Herramienta utilizada:

1. **Nombre:** Ruff
2. **Versión:** (0.12.4)
3. **Sitio oficial:** <https://docs.astral.sh/ruff/>

Descripción de la herramienta:

Ruff es una herramienta rápida de linting para proyectos Python que combina múltiples funcionalidades en un solo ejecutable: detección de errores comunes, verificación de estilo (similar a **flake8**), formateo (similar a **black**), y más. También es altamente configurable y significativamente más rápida que otros linters tradicionales.

Justificación de uso:

Ruff fue elegida por su velocidad, su compatibilidad con las reglas más comunes de PEP8, su capacidad de aplicar correcciones automáticas, y su facilidad de integración en flujos de trabajo modernos de desarrollo. Además, al funcionar como reemplazo de múltiples herramientas, simplifica el mantenimiento del entorno de desarrollo.

Configuración aplicada:

Se usó la configuración por defecto de Ruff, sin reglas personalizadas.

Resultados del análisis:

1. Se ejecutó **ruff check** . y se detectaron **72 errores** en total.
2. Se aplicó **ruff format** . para formatear automáticamente el código, lo que redujo los errores a **52**.
3. Se ejecutó **ruff check --fix** y se corrigieron automáticamente **51 errores**, quedando **1 error pendiente**.

4. El error restante fue corregido manualmente.
5. Finalmente, al volver a ejecutar `ruff check .`, **no se encontraron errores ni advertencias**.

Evidencia de ejecución:

- El archivo de log generado se encuentra en la carpeta `/logs/ruff.log`
- También puede revisarse la salida de `ruff check` y `ruff check --fix` en los archivos correspondientes.

Pantallazos:

```
[jm@T480]$ruff format .  
74 files reformatted, 5 files left unchanged
```

```
[jm@T480]$ruff check . --fix  
tests/test_julassom_CU17.py:10:1: E402 Module level import not at top of file  
|  
8 | sys.path.insert(0, project_root)  
9 |  
10 | from use_cases.CU17_search_books_screen import SearchBooksScreen  
| ~~~~~ E402  
  
Found 51 errors (50 fixed, 1 remaining).
```

```
tests/test_julassom_CU17.py:10:1: E402 Module level import not at top of file  
|  
8 | sys.path.insert(0, project_root)  
9 |  
10 | from use_cases.CU17_search_books_screen import SearchBooksScreen  
| ~~~~~ E402  
  
Found 1 error.
```

```
[jmc@T480]$ruff check .  
All checks passed!
```

Reporte de Calidad y Seguridad de Código

Análisis de Seguridad - *Bandit*

Herramienta: Bandit 1.8.6

Resultado general: Se identificaron múltiples advertencias, principalmente en dependencias externas del entorno virtual (`.venv`). No se encontraron vulnerabilidades directamente en el código fuente del proyecto.

Principales hallazgos:

- Uso de `assert` en múltiples librerías (e.g. `coverage`, `distutils`, etc.). Aunque su gravedad es baja, se recomienda evitar su uso en producción ya que puede eliminarse en bytecode optimizado.
- Uso de `eval()` y `exec()` en dependencias (riesgo medio), considerados potenciales vectores de inyección de código.
- Lectura de archivos temporales en `/tmp` — riesgo medio por uso de rutas temporales codificadas.
- Control unicode malicioso (`\u202e`) encontrado en `trojansource.py`. Este tipo de ataque puede ocultar código ejecutable visualmente (riesgo alto).
- SQL dinámico con expresiones concatenadas — posibles vectores de inyección SQL en dependencias (`coverage/sqldata.py`).

Nota importante: Todas las vulnerabilidades encontradas están localizadas en el entorno virtual (`.venv`). No se detectaron riesgos directos en el código de aplicación desarrollado por el equipo.